

Cellular Architecture Control Plane — System Design Study Guide

Session: SD1, 2026-09-27. Score: 7/10.

1. The problem

A multi-tenant SaaS serving ~100k requests/sec across ~50k tenants. The fleet is partitioned into **cells** — each cell a full, independent deployment of the serving stack handling a slice of tenants. Design the **control plane**: the system that assigns tenants to cells, routes requests to the right cell, detects unhealthy cells and evacuates them, and rolls out config and software across cells safely.

Targets: serving p99 < 200ms end-to-end, cell-routing lookup < 5ms, 99.99% serving uptime, control plane 99.9%.

2. Core concept: cell-based architecture

A cell is a bulkhead. Instead of one giant fleet where a bad deploy or a traffic spike can take down every tenant, you partition tenants across N independent cells. A failure is contained to the cells it touches — the blast radius is bounded by design.

Key properties of a cell: - Runs the full serving stack (not a shard of one layer). - Has a known capacity envelope (tenants per cell, rps per cell). - Fails independently — no shared fate with other cells (separate deploy, separate state, ideally separate failure domains).

The control plane exists because cells don't manage themselves: somebody has to decide which tenant lives where, notice when a cell is sick, move tenants, and ship software to all cells without recreating the fleet-wide outage cells were invented to prevent.

3. Control plane vs. data plane — the uptime gap

The single most load-bearing decision in the design:

- **Data plane** (serving path): must hit 99.99%.
- **Control plane** (assignments, health, rollouts): 99.9% is fine.

That gap is deliberate, and it dictates the architecture: **the serving path must never block on the control plane**. If every request needs a control-plane lookup, serving inherits the control plane's 99.9% — you've thrown away a nine.

Concretely: - The tenant→cell mapping is resolved at the edge/router tier from a **locally cached, versioned** copy. Lookup is a memory read, < 5ms, no network call. - The control plane's job is to *change* the mapping (reassignments, evacuations, new cells), not to *serve* it per request. - When the control plane is down, serving continues on the last-known-good mapping. Stale routing for a few minutes beats no routing at all.

4. The tenant→cell mapping: source of truth

Three layers, each with a distinct consistency contract:

1. **Source of truth: a strongly consistent store** (etcd/ZooKeeper-style — CP, not AP). The mapping is small (50k entries) and changes rarely, so you can afford consensus-grade writes. This is where "who owns the truth" lives. A DHT is the wrong tool here: membership shifts and fuzzy "all holders" semantics can't support the ack discipline a tenant move needs.
2. **Distribution: control plane pushes invalidations** (or version bumps) to routers; routers pull the full mapping on version mismatch or cold start. Push for speed, pull for recovery — a router that missed pushes or restarted converges on its own.
3. **Serving: routers hold the mapping in memory**. Cold-start behavior must be defined: serve nothing until the first mapping load completes (fail fast, not wrong).

During the propagation window, two routers can legitimately disagree about a tenant's cell. The design must tolerate that transiently (see §8 on moves).

5. Invariants (write these on the whiteboard first)

1. **A request is routed to exactly one cell** — the cell the router's mapping says owns the tenant. No fan-out, no "try both."
2. **Never assign a tenant to an evacuated cell**.
3. **An evacuated cell must reject tenant traffic even if it looks healthy**. Health signals from evacuated cells are informational only; re-admission is an explicit control-plane action, never automatic.
4. **Mid-move, a tenant has exactly one writer**. No new assignment for a tenant until the in-flight move is fully acked and propagated.

6. Health checking: liveness is not readiness

Two signals, because one lies:

- **Pings/heartbeats** → liveness. The process is alive. Cheap, frequent.
- **Last-successful-write timestamp per cell** → readiness. The cell is actually doing useful work.

The second signal catches the **zombie cell**: heartbeating fine, failing every write (disk full, deadlock, poisoned deploy). The master (see §7) polls each cell's write-freshness; `now - last_success > threshold` declares the cell unhealthy even with perfect pings. A ping-only health checker will happily route traffic into a dead-but-breathing cell.

7. The master and the move protocol

Tenant moves are gated by a leader-elected **master** in the control plane — the single writer that serializes reassignments. (The master itself needs leader election; that's the control plane's own availability story, and it only needs 99.9%.)

Planned move (old cell healthy and cooperating): 1. Master marks the tenant "moving" in the source of truth. 2. Write-lock on the old cell; incoming writes are queued, not rejected. 3. State drains/replicates to the new cell. 4. Master flips the mapping, pushes the invalidation to routers. 5. Routers ack; once the full ack set is in, the move is marked done and the lock releases. 6. Timeout + bounded retries on the ack phase — a move that can't complete in N tries fails loudly (error to the operator/tenant) rather than hanging forever.

Mid-move tenant experience: reads served from the old cell, writes queued and replayed in order. Queued writes will blow the p99 latency budget for that tenant — acceptable because moves are rare and the alternative (rejecting writes) is worse.

8. Unplanned failover: the dead cell

The case cells exist for. The old cell is unresponsive: no lock, no queue, no acks possible.

1. Master declares the cell dead (missed pings *and* stale write-freshness).
2. Tenants are reassigned to their **buddy cells** (see §9); mapping flips; routers converge.
3. RPO = replication lag of the shared log; RTO = detection time + reassignment + router propagation (minutes).

The nightmare is the **resurrection**: the dead cell reboots with stale data and starts serving before anyone tells it not to.

9. Fencing: making resurrection safe

Policy ("the cell must check before serving") is not a mechanism. The enforcement:

1. **Quarantine on boot.** A freshly booted cell's serving port is reachable by the control plane only. General traffic stays firewalled off. The cell cannot answer a single tenant request until re-admitted. Fail-closed: if the master is down, nothing gets re-admitted — safe direction.
2. **Version/epoch check.** The cell compares its latest write timestamps (per tenant) against the master's. Mismatch = stale data.
3. **Catch-up.** The cell replays the tenant's topic from the shared log (§10) from its last known offset until it's current. (Alternative when the data is hopelessly stale: wipe the cell and keep it as empty reserve capacity for new tenants — often the cleanest answer.)
4. **Re-admission.** Master issues a fresh epoch/lease; only then does the port open to routers. Routers stamp requests with the current epoch and ignore responses from stale epochs — so even a leaked stale response can't corrupt anything.

10. Replication model: buddy cells + shared log

The committed tradeoff ($99.99\% \approx 52.6$ minutes of downtime per year — worth paying for):

- **Every cell has a buddy in a separate region** holding a live copy. (Same-region buddy only if a customer demands it and pays.)
- **Writes go to a shared, durable log** (Kafka-like), not a per-cell queue — a per-cell queue shares the cell's fate and dies with it. Tenants map to topics.
- Replication to the buddy is **asynchronous** via the log: low write latency, but RPO = shipment lag (seconds), not zero. Say the number out loud.
- The buddy consumes the tenant's topic and applies writes continuously, so failover is a mapping flip, not a restore.

Cost: ~2x storage plus the cross-region log traffic. What you buy: RPO of seconds, RTO of minutes, no backup-restore path on the critical failover.

11. Rollout: never fleet-wide

A bad config or binary pushed everywhere at once is a fleet-wide outage — exactly what cells prevent. Progressive delivery:

1. **Dev → preprod** (synthetic + mirrored traffic).

2. **Canary slice:** cells serving similar tenants (controls for tenant-type variance) — small blast radius.
3. **10% → 50% → 100%** of cells over days, with bake time at each stage.
4. **Promotion gates per stage:** error rate and p99 latency vs. the pre-rollout baseline; deployment verification scripts must pass.
5. **Auto-rollback triggers:** error spike, cell-death rate alarming, verification failure. Rollback = previous version push, same staged path in reverse (fast path for config).

Note the ordering mistake to avoid: starting at "a region" is not a canary — a region is thousands of tenants.

12. Observability

Dashboard: - Per-cell health: ping status, write-freshness, tenant count, rps, p99 latency. - Read/write request health per cell and fleet-wide. - Rollout state: which version/config each cell is on, stage progress, bake timers. - Mapping version distribution across routers (who's stale?).

Paging: - **Sev-2 (wake a human):** cell-death rate alarming — cells missing pings *or* write-freshness going stale. - Everything else (single-cell evacuation, staged rollout progress, a move timing out and failing loudly) is handled by automation and reviewed in business hours.

13. The ideal architecture, end to end

Edge routers with a versioned, in-memory tenant→cell map (source of truth: etcd-like CP store; control plane pushes invalidations, routers pull on mismatch). Independent cells, each with a cross-region buddy; writes flow through a shared Kafka-like log with per-tenant topics, buddies consume continuously. Health is pings plus per-cell write-freshness, watched by a leader-elected master. Moves: planned = write-lock + drain + master-acked two-phase with timeouts; unplanned = declare dead, fence via quarantine-on-boot and epochs, flip mapping to buddy. Rollout is dev → preprod → canary slice → 10/50/100 with bake gates and auto-rollback. Dashboard shows cell health, write freshness, and rollout state; sev-2 pages on cell-death rate.

14. Session notes: what went right / what to fix

Went right: the control/data-plane separation was immediate and unprompted; real invariants stated early; committed to buddy replication with the 52-minutes-per-year math; Kafka log broke fate-sharing; zombie detection via write-freshness; quarantine-on-boot fencing; took every counterexample within one round.

To fix: name the CP store directly instead of hiding it behind "master"; pin the RPO number the async queue implies; get from policy to enforcement mechanism faster on fencing; commit to the replication model without hedging; canary means smallest blast radius first, not a region; size the fleet (cell count × tenants per cell) before designing around it.

Redo drills: (1) state the mapping store's consistency choice in the first two minutes; (2) RPO/RTO numbers alongside every replication decision; (3) boot-sequence fencing as a mechanism, drawn as a sequence.